0038洛谷P5731 蛇形方阵

```
#include<iostream>
#include<iomanip>
using namespace std;

int main() {
    int n;
    cin >> n;

    int a[15][15] = { 0 };
```

先建立一个二维数组用来表示地图上的坐标

```
    int dx[4] = { 0,1,0,-1 };
    int dy[4] = { 1,0,-1,0 };
```

设置两个数组，用“令下一x，y值分别为当前x,y值分别加上dx，dy值”的方法
使之拥有控制输出方向的功能

d分别为0,1，2,3时，(dx,dy)分别为向右(0,1)`向下(1,0)`向左(0,-1)`向上(-1,0)，这正好符合顺时针

```
    int x = 0, y = 0;
    int d = 0;
    int num = 1;
```

初始化整数num作为当前输出的数字，先初始化它为0

```
    while (num <= n * n) {
        a[x][y] = num;
```

将num存入a[][]中

```
        num++;
```

每存入一个数令num++

```
        int nx = x + dx[d];
        int ny = y + dy[d];
```

注意看，这里设置了一个nx和一个ny，为什么不直接令x+=dx（x=x+dx的精简写法），y+=dy呢

```
        if (nx < 0 || nx >= n || ny < 0 || ny >= n || a[nx][ny] != 0) {
```

判断nx,ny是否会越界

```
            d = (d + 1) % 4;
            nx = x + dx[d];
            ny = y + dy[d];
        }
```

因为这串写法中，我们提供了一个nx和ny来判断是否越界，一旦越界，就改变nx，ny的朝向（也就是改变d值）

而如果我们直接写x+=x,y=+y，当我们判断x出界时，就不太方便更改轨道（这题理论上也是可以修改回来的）

可以粗糙理解为先设置一个假的x和假的y来进行下一步操作，当操作没有越界时就令该操作为正式操作，操作越界时更改方向

这是一个很好用的防越界算法

```
        x = nx;
        y = ny;
    }
    for (int i = 0;i < n;i++) {
        for (int j = 0;j < n;j++) {
            cout << setw(3) << a[i][j];
```

setw（3）可以控制输出的字宽，填不下的默认是在数值前用空格来填

```
        }
        cout << endl;
    }
    return 0;
}
```

完整代码

```
#include<iostream>
#include<iomanip>
using namespace std;

int main() {
    int n;
    cin >> n;

    int a[15][15] = { 0 };

    int dx[4] = { 0,1,0,-1 };
    int dy[4] = { 1,0,-1,0 };

    int x = 0, y = 0;
    int d = 0;
    int num = 1;

    while (num <= n * n) {
        a[x][y] = num;

        num++;

        int nx = x + dx[d];
        int ny = y + dy[d];

        if (nx < 0 || nx >= n || ny < 0 || ny >= n || a[nx][ny] != 0) {

            d = (d + 1) % 4;
            nx = x + dx[d];
            ny = y + dy[d];
        }

        x = nx;
        y = ny;
```

```
    }
    for (int i = 0;i < n;i++) {
        for (int j = 0;j < n;j++) {
            cout << setw(3) << a[i][j];

        }
        cout << endl;
    }
    return 0;
}
```